

Missouri University of Science and Technology

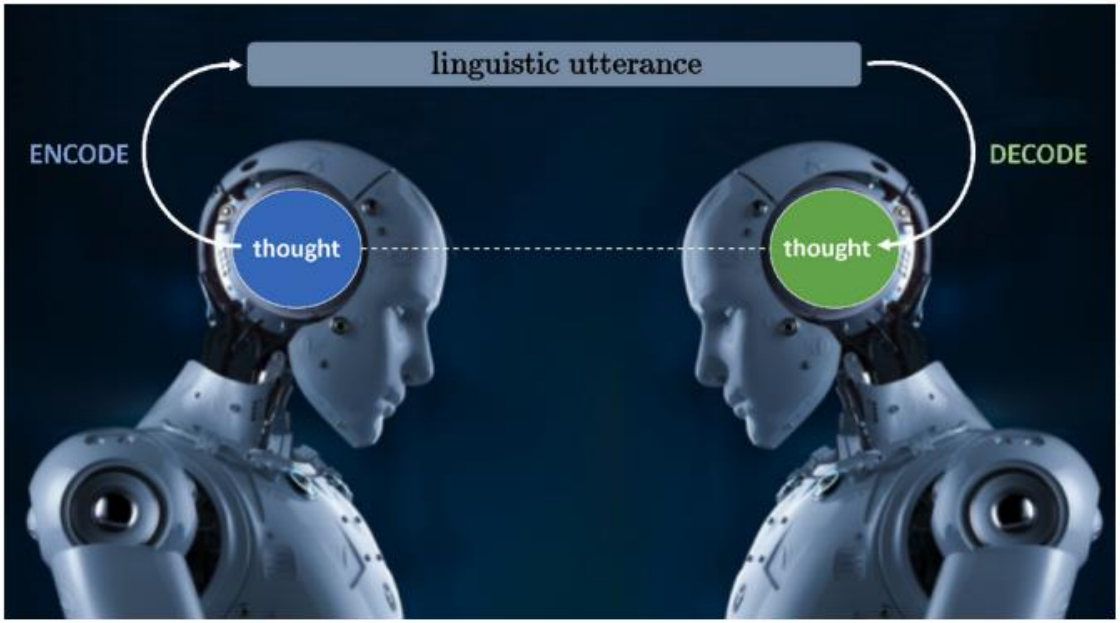
Intro to Data Science
CS 3402 (Spring 2026)

Lecture 13 Basics of Natural Language Processing

Xiaowei Yu, Ph.D.

Department of Computer Science

Natural Language Processing



The gradient, Walid S. Saba

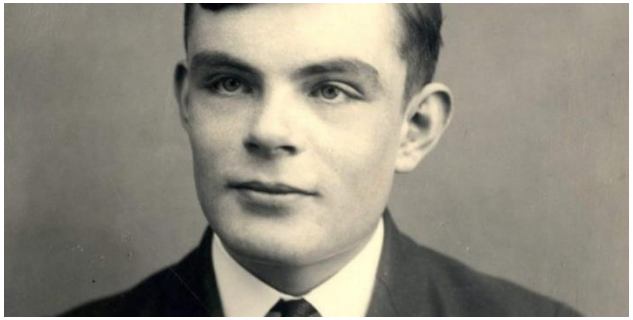
Natural language processing is the set of methods for making human language accessible to computers

(Jacob Eisenstein)



History of NLP - Turing Test

“Computing Machinery and Intelligence” Mind, Vol. 59, No. 236, pp. 433-460, 1950



I propose to consider the question "Can machines think?"... We can only see a short distance ahead, but we can see plenty there that needs to be done.

The Turing test, proposed by Alan Turing in 1950, is a measure of machine intelligence that determines if a computer can exhibit behavior indistinguishable from a human.

History of NLP - ELIZA

ELIZA was an early natural language processing system capable of carrying on a limited form of conversation with a user.

Developed by MIT (1960s) that simulated conversation by rephrasing user input using pattern matching



History of NLP – (1950 – 1970)

Mid 1950's – Mid 1960's – Birth of NLP and Linguistics

- At first, people thought NLP is easy! Researchers predicted that "machine translation" can be solved in 3 years or so
- Mostly hand- coded rules / linguistic -oriented approaches
- The 3-year project continued for 10 years, but still no good result, despite the significant amount of expenditure

Mid 1960's – Mid 1970's – A Dark Era

- After the initial hype, a dark era follows
- People started believing that machine translation is impossible, and most abandoned research for NLP

History of NLP – (1970 – 2000)

1970's and early 1980's – Slow Revival of NLP

- Some research activities revived, but the emphasis is still on linguistically oriented, working on small toy problems with weak empirical evaluation

Late 1980's and 1990's – Statistical Revolution!

- By this time, the computing power increased substantially
- Data- -driven, statistical approaches with simple representation win over complex hand-coded linguistic rules
- "Whenever I fire a linguist, our machine translation performance improves." (Frederick Jelinek, IBM, 1988)

2000's – Statistics Powered by Linguistic Insights

- With more sophistication with the statistical models, richer linguistic representation starts finding a new value

History of NLP – (2000 – Current)

2010's – Emergence of embedding model and deep neural networks

- Several embedding models for text using neural networks and deep neural networks were proposed including Word2Vec, Glove, fastText, Elmo, BERT, COLBERT, GPT1, GPT2
- New techniques brought attention to more complex tasks

2020's – The Era of Large Language Models (LLMs)

- Emergence of **foundation models** trained on massive web-scale corpora, ChatGPT, Gemini, Claude, DeepSeek, LLaMA, Qwen, etc.
- Scaling laws: bigger models + more data + more compute → better performance
- Pretraining + fine-tuning paradigm becomes dominant
- Shift from task-specific models → general-purpose language models

2026 – Major U.S. technology companies are projected to invest over $700 billion in 2026 alone betting on artificial general intelligence (AGI)

History of NLP – (2026 –)

What will be the future of NLP?

1. From "Chatbots" to "Agentic Workflows”
2. Physical AI, Vision-Language-Action Models, “World Models”

…

Natural Language Processing

Natural Language Processing (NLP) is the field that enables computers to:

- Understand human language
- Process text data
- Extract meaning from text

Applications:

- Sentiment analysis
- Spam detection
- Chatbots
- Machine translation
- Search engines

NLP - Sentiment analysis

Sentiment Analysis
Sentiment analysis = identifying the attitude expressed in text.

We classify text into:

Positive
Negative

Example:
"This movie was fantastic!" → Positive
"I hated this film." → Negative


- unbelievably disappointing


- Full of zany characters and richly applied satire, and some great plot twists


- this is the greatest screwball comedy ever filmed

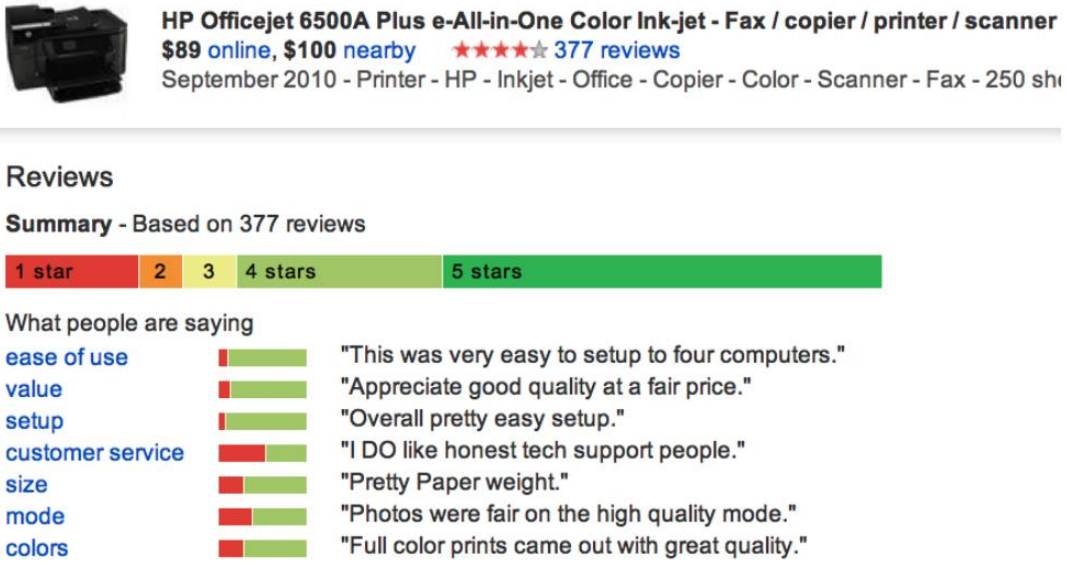

- It was pathetic. The worst part about it was the boxing scenes.

We focus on **binary classification**

NLP - Sentiment analysis

Sentiment Analysis

Sentiment analysis = identifying the attitude expressed in text.



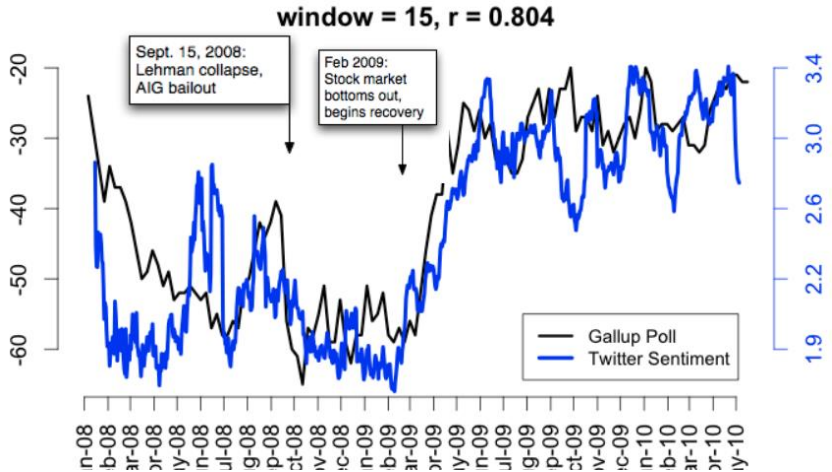
NLP - Sentiment analysis

Sentiment Analysis

Sentiment analysis = identifying the attitude expressed in text.

Twitter sentiment versus Gallup Poll of Consumer Confidence

Brendan O'Connor, Ramnath Balasubramanyan, Bryan R. Routledge, and Noah A. Smith. 2010.
From Tweets to Polls: Linking Text Sentiment to Public Opinion Time Series. In ICWSM-2010



NLP - Sentiment analysis

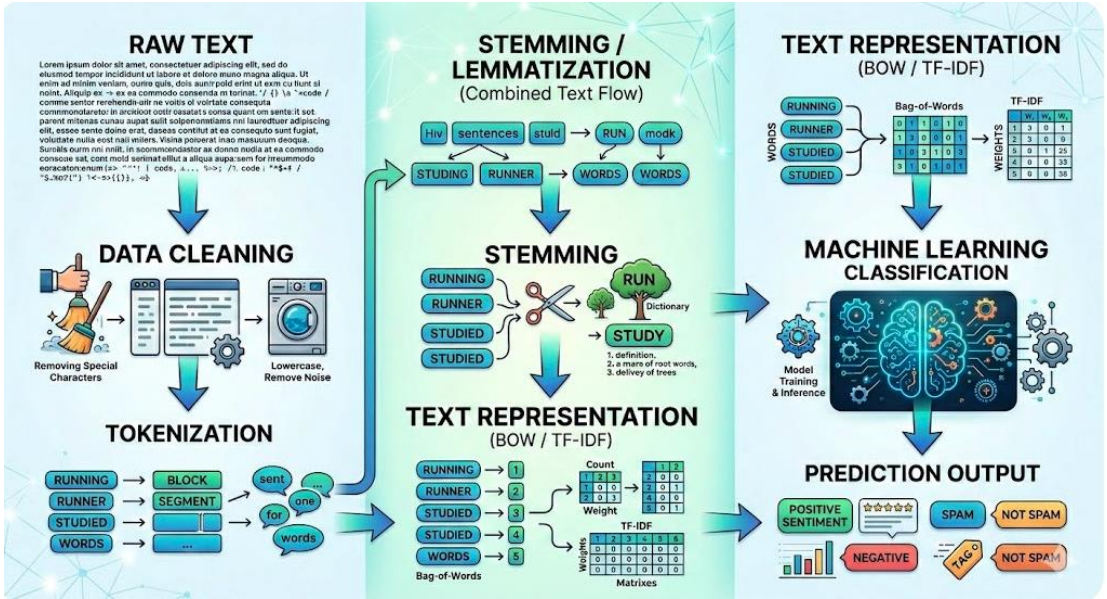
Sentiment analysis has many other names

- Opinion extraction
- Opinion mining
- Sentiment mining
- Subjectivity analysis

NLP Pipeline Overview

Basic NLP Pipeline: Text → Clean → Tokenize → Normalize → AI Model → Output

data | data preprocessing | model | results



Data Cleaning – Remove noise;

Tokenization – Split text into words/tokens;

Text Representation – Convert text to numerical vectors;

Classification – Train model to predict labels;

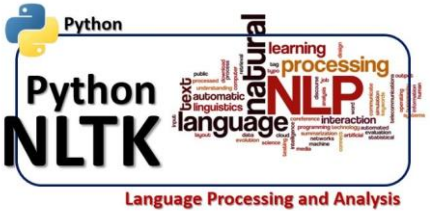
NLP Pipeline - Data Cleaning & Tokenization

raw_docs = [“The point of this example is to _learn how basic text \ cleaning works_ on *very simple* data."]

NLTK **(Natural Language Toolkit)** makes it easy to convert documents-as-strings into word-vectors, a process called **tokenizing**.

['The', 'point', 'of', 'these', 'examples', 'is', 'to', '_learn', 'how', 'basic', 'text', 'cleaning', 'works_', 'on', '*', 'very', 'simple', '*', 'data', '.']

Colab Practice of Tokenization using NLTK.



NLP Pipeline - Data Cleaning & Tokenization

raw_docs = [“The point of this example is to _learn how basic text \ cleaning works_ on *very simple* data."]

Tokenizers:

- Cl100k_base (GPT-4, GPT-3.5) by OpenAI
- O200k_base (newer GPT-4o family) by OpenAI
- GemmaTokenizer by Google DeepMind
- LlamaTokenizer by Meta AI
- …..

NLP Pipeline - Data Cleaning & Tokenization

raw_docs = [“The point of this example is to _learn how basic text \ cleaning works_ on *very simple* data."]

Colab Practice of Tokenizer from OpenAI. tokenizer = AutoTokenizer.from_pretrained("gpt2")

Text: The point of these examples is to _learn how basic text cleaning works_ on *very simple* data.

Tokens: ['The', 'Ġpoint', 'Ġof', 'Ġthese', 'Ġexamples', 'Ġis', 'Ġto', 'Ġ_', 'learn', 'Ġhow', 'Ġbasic', 'Ġtext', 'Ġcleaning', 'Ġworks', '_', 'Ġon', 'Ġ*', 'very', 'Ġsimple', '*', 'Ġdata', '.']

NLP Pipeline - Removing Punctuation

- Removing them helps **clean the text** and reduces noise
- Common approach: use **regular expressions** or **string** libraries
- Punctuation symbols usually **do not carry sentiment meaning**

“I love AI, it‘s amazing!” vs. "I love AI its amazing"

Colab Practice of Removing Punctuation using String Library.
['The', 'point', 'of', 'these', 'examples', 'is', 'to', '_learn', 'how', 'basic', 'text', 'cleaning', 'works_', 'on', 'very', 'simple', 'data']

NLP Pipeline - Text Representation

Having a representation of the text that has been cleaned.

- Machine learning models **cannot read raw text**
- Text must be **converted to numerical vectors**
- Enables **classification, clustering, and similarity computation**

Text → “Bag of Words” → Feature vector

NLP Pipeline - Bag of Words

- Count **how often each word appears** in a document
- Each word becomes a **dimension** in a feature vector
- Simple but effective for **basic sentiment analysis**

Text 1: "I like tomatoes more than apples"
Text 2: "I like reading"



"Cleaned text → BoW vector (same feature space with same size) → ML model input"

NLP Pipeline - Bag of Words

TF-IDF: a particular case of bag of words

- Not all words are equally important.
- **TF (Term Frequency):** how often a word appears in a document
- **IDF (Inverse Document Frequency):** downweights words that appear in **many documents**
- TF-IDF = TF × IDF

NLP Pipeline - Bag of Words

Count the terms per document, which is the term frequency vector.

TF (Term Frequency)

```
mydoclist = ['Mireia loves me more than Hector loves me',
    'Sergio likes me more than Mireia loves me',
    'He likes basketball more than football']
from collections import Counter
for doc in mydoclist:
    tf = Counter()
    for word in doc.split():
        tf[word] += 1
    print(tf.items())
```

```
[('me', 2), ('Mireia', 1), ('loves', 2), ('Hector', 1),
('than', 1), ('more', 1)] [('me', 2), ('Mireia', 1), ('likes',
1), ('loves', 1), ('Sergio', 1), ('than', 1), ('more', 1)]
[('basketball', 1), ('football', 1), ('likes', 1), ('He', 1),
('than', 1), ('more', 1)]
```

NLP Pipeline - Bag of Words

```
mydoclist = ['Mireia loves me more than Hector loves me',
    'Sergio likes me more than Mireia loves me',
    'He likes basketball more than football']
```

Vocabulary vector is [me, basketball, Mireia, football, likes, loves, Sergio, Hector, He, than, more]$_{1\times 11}$

1st doc: 'Mireia loves me more than Hector loves me'.

TF (Term Frequency) vector for 1st doc is [2, 0, 1, 0, 0, 2, 0, 1, 0, 1, 1]

2nd doc: Sergio likes me more than Mireia loves me'.

TF (Term Frequency) vector for 2nd doc is [2, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1]

TF (Term Frequency) vector for 3rd doc?

NLP Pipeline - Bag of Words

Vector Normalization: L2 Normalization

$$\hat{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_2} = \frac{\mathbf{x}}{\sqrt{\sum_{i=1}^{n} x_i^2}}$$

TF (Term Frequency) vector for 1st doc is [2, 0, 1, 0, 0, 2, 1, 0, 0, 1, 1]

Normalized TF vector for 1st doc is [0.57735027, 0, 0.28867513, 0, 0, 0.57735027, 0.28867513, 0, 0, 0.28867513, 0.28867513]

Scale down the vectors so that each element is between [0, 1]

Tokenization

https://colab.research.google.com/drive/1OxbkFDBBLSHdhVVS33YYT_rofrqvYNF9?usp=sharing

Modern Token Vectorization (LLMs)

Early NLP:

- Bag of Words
- TF-IDF

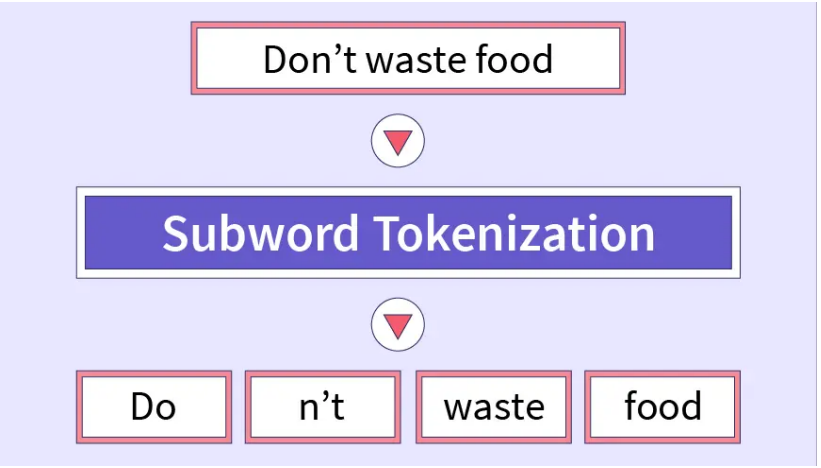
Modern Large Language Models (LLMs):

- Subword tokenization
- Dense embeddings

Modern Token Vectorization (LLMs)

Modern LLMs Do NOT Operate on Full Words

Instead of splitting text into whole words, modern LLMs use **subword tokenization**.



Modern Token Vectorization (LLMs)

Why use subwords?

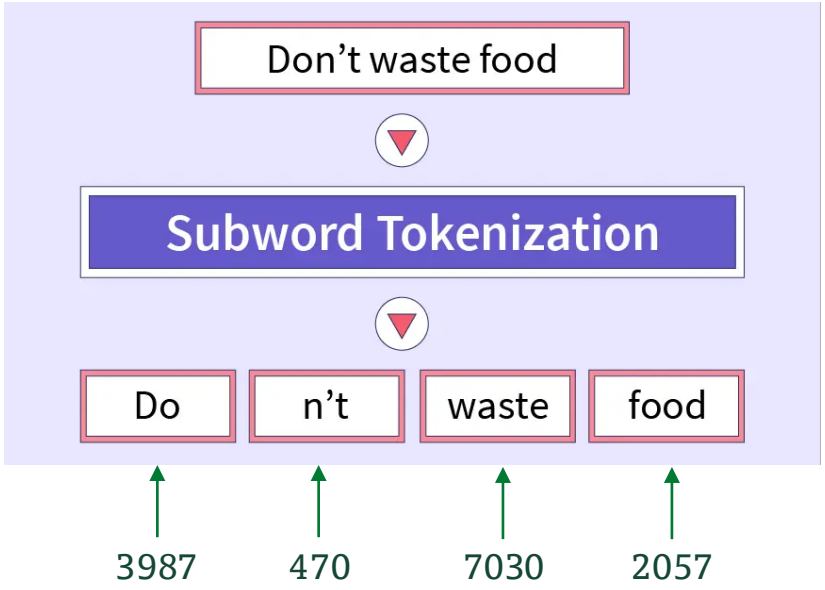
1. Handle rare words
   - Instead of storing every rare word, the model builds it from subwords.
2. Handle new (unseen) words
   - Example: “bioinformaticsAItool”. Can be broken into known subwords.
3. Reduce vocabulary size
   - Instead of millions of words, vocabulary ≈ 30k–50k tokens.

Modern Token Vectorization (LLMs)

After tokenization:
Text
→ Tokens
→ Token IDs

Instead of millions of words, vocabulary ≈ 30k–50k tokens.

The model works with **token IDs**, NOT raw text.



From GPT2 Tokenizer

Modern Token Vectorization (LLMs)

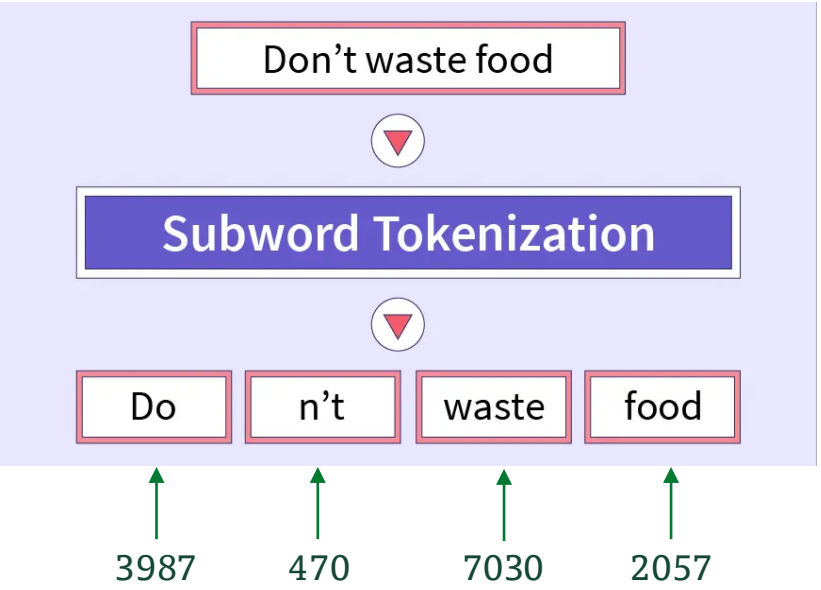
Token → Embedding Vector

Text
→ Tokens
→ Token IDs
→ **Embedding Vectors**

Token Embeddings Shape: [1, 4, 768] (GPT2 Tokenizer)

```
tensor(
[[[ 0.0473, 0.0353, 0.1842, ..., 0.0602, -0.0351, 0.0087],
[ 0.0334, -0.0659, 0.0658, ..., -0.2512, -0.2332, 0.1274],
[ 0.0512, 0.0066, 0.1134, ..., -0.0295, -0.1906, 0.1587],
[ 0.0517, 0.0081, 0.0174, ..., 0.0501, -0.0998, 0.0025]]],
grad_fn=<EmbeddingBackward0>)
```

Trainable, real numbers, dense



From GPT2 Tokenizer

MLP Coding Practice

https://colab.research.google.com/drive/1wbOaBvac9zyt5Ho0wPUGcr94A1M1I4na?usp=sharing

